

Comunicación simultánea y disponibilidad del servicio

4. Comunicación simultánea y disponibilidad del servicio

Comunicación simultánea y disponibilidad del servicio

Hasta ahora has comprobado que tu API responde bien a una petición aislada: el código de estado correcto, el JSON esperado, el test que pasa. Hoy cierras el tema con dos preguntas distintas, sobre ese mismo servicio ya en marcha: ¿aguanta varias peticiones a la vez sin que unas tengan que esperar a otras? Y, más allá de una petición concreta, ¿cómo sabes si el servicio en general sigue sano y disponible ahora mismo, sin comprobarlo tú a mano?

Comunicación simultánea de varios clientes

Piensa en tu propio GameVault, ya en producción: no lo vas a usar solo tú. Si veinte personas consultan el catálogo a la vez y tu servidor las atendiera de una en una —terminar la primera antes de empezar la segunda—, la última persona esperaría veinte veces más que la primera.

Spring Web, sobre Tomcat, atiende cada petición HTTP en un hilo distinto —una línea de ejecución independiente que avanza en paralelo con las demás— tomado de un pool ya creado de antemano. Es como varios camareros, cada uno con su propia mesa, en vez de un único camarero que no pasa a la siguiente mesa hasta terminar con la anterior.

Un hilo por petición, tomado de un pool



Cada petición la procesa un hilo ya creado de antemano en el pool, no uno nuevo cada vez (verás ambas ideas con detalle en el Tema 3).

Cómo lo vas a comprobar tú mismo

En la Actividad 1.4 montarás un método del service con un `Thread.sleep(2000)` puesto a propósito, que simula una consulta lenta, y lanzarás dos peticiones simultáneas contra su endpoint:

```
time (curl -s http://localhost:8080/api/v1/libros/top & \  
      curl -s http://localhost:8080/api/v1/libros/top & \  
      wait)
```

Secuencial vs. en paralelo

Antes de probarlo tú mismo, compara las dos formas posibles en que tu servidor podría atender esas dos peticiones:



Secuencial — un solo hilo

Petición 1 — 2s



Petición 2 — 2s



En paralelo — un hilo por petición

Petición 1 — 2s

Petición 2 — 2s

Qué significan esos tiempos

Si las dos peticiones se atendieran una detrás de otra (Secuencial), el conjunto tardaría unos 4 segundos. Si se atienden en paralelo, tardan aproximadamente 2 — porque cada una la procesa un hilo distinto del pool, igual que los dos camareros de la analogía.

Puedes confirmarlo añadiendo temporalmente una traza con `Thread.currentThread().getName()` en el método y mirando el log: verás dos nombres de hilo distintos (`http-nio-8080-exec-1`, `http-nio-8080-exec-2...`) para las dos peticiones.

Qué es la disponibilidad de un servicio

Todo lo anterior daba por hecho algo que no has cuestionado en ningún momento: que tu servicio está funcionando de verdad. ¿Cómo lo sabes, en realidad?

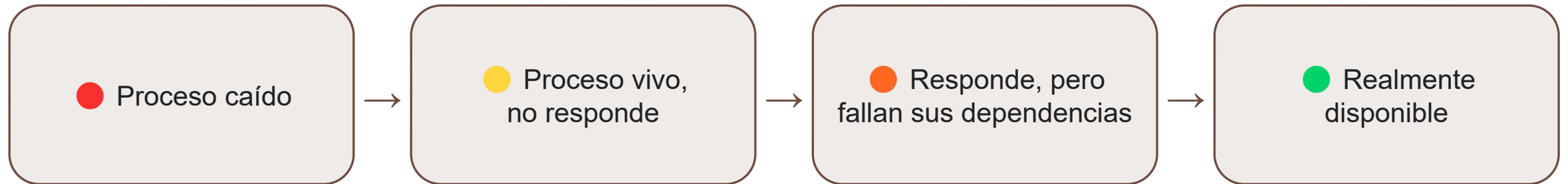
Imagina que tu GameVault, ya desplegado, se cae un sábado a las tres de la madrugada. Si nadie lo comprueba a mano, nadie se entera hasta que un usuario se queja de que la web no funciona — pueden pasar horas.

Distintos niveles de "estar bien"

Que un servicio esté disponible no es una única cosa binaria — hay distintos niveles, cada uno más exigente que el anterior:

- El proceso está arrancado: la aplicación no se ha caído, sigue viva.
- Responde peticiones: acepta conexiones y contesta algo, lo que sea.
- Sus dependencias funcionan: la base de datos, la cola de mensajería, cualquier servicio del que dependa, están accesibles — un proceso vivo que no puede hablar con su base de datos no está realmente "disponible" para hacer su trabajo.

De proceso caído a realmente disponible



Qué es un health check

Un health check (comprobación de salud) es una forma automática —pensada para que la ejecute una máquina, sin intervención humana— de responder a estas preguntas cada pocos segundos, sin que nadie tenga que mirar una pantalla.

Quien lo consulta no eres tú, a mano: puede ser un programa que avise a alguien en cuanto el servicio deje de responder, un sistema de despliegue que lo reinicie automáticamente, o el propio CI, que comprueba que arranca correctamente antes de darlo por bueno.

Spring Boot Actuator

Spring Boot Actuator es la implementación de todo esto para una aplicación Spring Boot: un conjunto de endpoints HTTP listos para usar que exponen información operativa sobre tu aplicación — entre ellos, su salud.

Tu propio proyecto no incluye Actuator todavía (revisa tu pom.xml: no está la dependencia) — es la mejora que añades esta semana:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

No aparecen en Swagger UI con la configuración actual

De forma predeterminada, springdoc no añade los endpoints de Actuator a Swagger UI. Por eso verás los endpoints de tu API, pero no `/actuator/health`.

Tiene sentido mantenerlos separados: `/actuator/health` no es un recurso del catálogo pensado para sus usuarios, sino información operativa para comprobar el estado de la aplicación. Consultarás `/actuator/health` directamente desde el navegador o mediante curl.

Ver el detalle de cada dependencia

Con solo esa dependencia, Spring Boot expone automáticamente `/actuator/health`. Para ver, durante el desarrollo, el detalle de cada dependencia —y no solo un UP o DOWN general— añade esto en `application-dev.yml`:

```
management:
  endpoint:
    health:
      show-details: always
```

show-details: always expone información

Permite que cualquiera que pueda acceder a `/actuator/health` vea el detalle de los componentes comprobados. En estas prácticas resulta útil porque trabajarás en local y podrás identificar rápidamente si ha fallado PostgreSQL, MongoDB u otra dependencia.

En producción, estos detalles normalmente se ocultan o se muestran únicamente a usuarios autorizados.

El endpoint /actuator/health

Con los detalles activados, agrega el estado de cada dependencia real que Spring Boot detecta en el classpath:

```
{
  "status": "UP",
  "components": {
    "db": { "status": "UP" },
    "mongo": { "status": "UP" }
  }
}
```


La implicación de agregar cada dependencia

Si el contenedor de MongoDB se cae, /actuator/health pasa a DOWN con el detalle del componente mongo marcado como el culpable — aunque la aplicación siga respondiendo peticiones sobre PostgreSQL sin ningún problema. Es exactamente el matiz de antes: "responder" y "estar realmente sano" no son lo mismo.

Pensado para máquinas, no para personas

`/actuator/health` no deja de ser una petición HTTP estándar — el mismo protocolo de siempre. La diferencia es quién lo consulta: normalmente no una persona con un navegador, sino un orquestador o un monitor, cada pocos segundos, de forma automática.

Actuator trae también otros endpoints útiles, como `/actuator/info` (metadatos de la aplicación) o `/actuator/metrics` (métricas de rendimiento) — no vas a profundizar en ellos ahora, pero conviene que sepas que existen.

- Cada petición HTTP la atiende un hilo distinto del pool de Tomcat — por eso dos peticiones lentas simultáneas no tardan el doble, sino aproximadamente lo mismo que una sola.
- La disponibilidad de un servicio tiene varios niveles: proceso vivo, responde peticiones, dependencias funcionando — no son lo mismo.
- Un health check es una comprobación automática, pensada para que la consulte una máquina (un programa de monitorización, un sistema de despliegue, el propio CI), no una persona.

Ideas clave (cont.)

- Spring Boot Actuator expone `/actuator/health` con la dependencia `spring-boot-starter-actuator`; `management.endpoint.health.show-details: always`, durante el desarrollo, permite ver el detalle de los componentes comprobados por Actuator.
- `/actuator/health` agrega el estado de cada dependencia real (PostgreSQL, MongoDB...) — si una cae, el estado general pasa a `DOWN` aunque el resto siga funcionando.
- Los endpoints de Actuator no aparecen en Swagger UI: `springdoc` solo escanea tus propios `@RestController`, no el mecanismo aparte que usa Actuator.